



Matrícula	Nome
-----------	------

Responda F (falso), V (verdadeiro) ou NS (não sei). Duas respostas erradas anulam uma certa. (0,4 cada).
Justifique as questões que responder como falso.

1. As razões do estudo comparativo de LP são: aumentar a capacidade para expressar conceitos, aumentar a capacidade de projetar novas linguagens, melhorar a capacidade de selecionar a linguagem mais adequada, permitir desenvolver programas mais eficientes levando em conta a implementação de recursos da linguagem.	V
2. LP para aplicações científicas possuem estruturas de dados simples, facilidade e precisão para cálculo, laços iterativos e seleções, facilidade para produzir relatórios elaborados e capacidade de lidar com aritmética BCD.	F
3. Os critérios de legibilidade, qualidade da definição e custos estão associados à produtividade. Confiabilidade, capacidade de escrita, portabilidade e generalidade estão associados à manutenibilidade de uma linguagem.	F
4. Linguagem compilada utiliza checagem de tipo estática e requer declaração de variáveis.	V
5. A introdução do protótipo de função na linguagem C pela ANSI permitiu a checagem de tipo dinâmica.	F
6. Arquitetura de Von Neumann é caracterizada por programa e dados na memória, periféricos, memória e CPU ligados por barramentos, ciclo busca-decodificação-busca de operandos-execução, e processamento sequencial.	V
7. No paradigma procedural (ou imperativo) o algoritmo de resolução é definido em detalhes, a ordem de execução dos comandos é definida, sendo admitida a existência de comando de atribuição.	V
8. No paradigma lógico a máquina de inferência escolhe a ordem de execução das regras quando houver um conjunto de conflito. Assim, não é necessário suporte para recursão direta ou indireta.	F
9. Em sistemas de interpretação pura, se não houver erros, o tradutor analisa o programa fonte e gera um código intermediário que é interpretado, por exemplo, Java que utiliza o compilador javac e a máquina virtual Java.	F
10. Uma linguagem que apresenta suporte para abstração de dados possui maior capacidade de escrita do que uma que possui suporte para abstração funcional.	NS
11. Uma das características de Hugs é ter funções globais tais como as construídas com as expressões let ou where .	F
12. Uma função em Hugs é definida como um conjunto de equações onde cada equação é uma regra de reescrita que pode conter guardas.	V
13. Seja a função zero definida como: zero x = 0 . Aplicando a regra de reescrita para avaliar zero (1/0) ocorrerá o seguinte erro: Program execution error: {primDivDouble 1.0/0.0}	F
14. Na avaliação de uma <i>query</i> Prolog quando há uma falha a máquina de inferência tenta satisfazer a meta com alternativas, buscando um casamento a partir de onde a meta falhou, realizando busca em abrangência.	NS
15. Pela hipótese do mundo fechado se um fato não é conhecido ser verdadeiro, assume-se que ele é falso.	V
16. Uma expressão em Hugs é resolvida utilizando-se a regra de redução que substitui a expressão corrente pela expressão do lado direito, conforme os padrões casados com a instância de argumentos.	V
17. Pela avaliação preguiçosa, uma expressão só é avaliada quando seu valor é requerido e se for cotizada, aparecendo em vários lugares, só é avaliada uma única vez em Prolog.	F
18. O ciclo de execução de Prolog aplica regra de resolução com estratégia linear, na ordem de escrita das cláusulas, com encadeamento de regras para a frente, busca em profundidade, da esquerda para a direita das condições das cláusulas, backtracking quando a unificação falhar, e sem occur-check na unificação.	F
19. A função $((^3) \cdot (/2)) \ 10$ é avaliada como 500.	F
20. A função <code>let fiblst = 1:1:[x+y (x,y) <- (zip fiblst (tail fiblst))]</code> in <code>take 5 fiblst</code> é avaliada como <code>[1,2,3,5,8]</code>	F



Matrícula

Nome

Resolva apenas a questão 21 ou a 22. Se for detectada cola, será atribuída a nota zero.

Questão 21 – Explique todo o processo de resolução de uma *query* Prolog (1,0). Construa um programa que contenha um *cut* e uma *query* Prolog (0,5) e utilize uma árvore de resolução para ilustrar o processo de avaliação da *query* (1,5). Indique quais subárvores não serão percorridas (0,5). Não use exemplos da sala de aula. [3,5]

R: Na resolução de uma *query* Prolog, são buscadas primeiramente as informações na base de conhecimento, linha a linha, na ordem que foram acrescentadas. Por exemplo, caso seja pesquisado por primo(joao, maria), e na BC tiver tal informação, a *query* retornará true e continuará a execução a partir da linha seguinte caso seja requisitado pelo ponto e vírgula.

Para a ilustração da árvore de resolução, estarei realizando um programa que contém na BC dados de namorados (X namora Y, implicando em Y namora X) e dados de traições (X traiu Y), com o objetivo de encontrar pessoas que traem ou que são traídas.

Estarei colocando que Mateus namora Rebeca (e portanto Rebeca namora Mateus), João namora Emanuely, e Xavier namora Bárbara, e que João trai Emanuely.

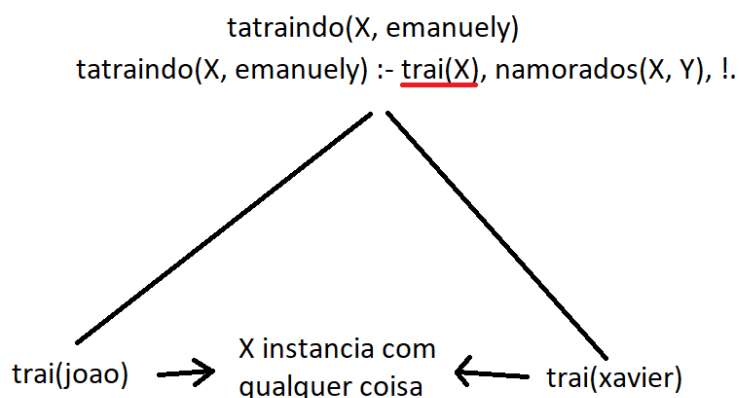
Para não precisar incluir as implicações (por exemplo, namora(rebeca, mateus)), criarei a regra namorados(X,Y) que retorna true caso X seja namorado de Y ou vice-versa: `namorados(X,Y) :- namora(X,Y); namora(Y,X)`

E a função que desejamos obter, `tatraindo(X,Y)`, nos dirá se X está traindo Y. Para isso ser verdade, X deve namorar com Y, e X deve estar na lista de traições. Portanto, BC:

```
namora(mateus, rebeca).  
namora(joao, emanuely).  
namora(xavier, barbara).
```

```
trai(joao).  
trai(xavier).  
namorados(X,Y) :- namora(X,Y); namora(Y,X).  
tatraindo(X,Y) :- trai(X), namorados(X,Y), !.
```

Árvore de resolução para `tatraindo(X, emanuely)` (gostaríamos de saber se existe X que trai Emanuely):

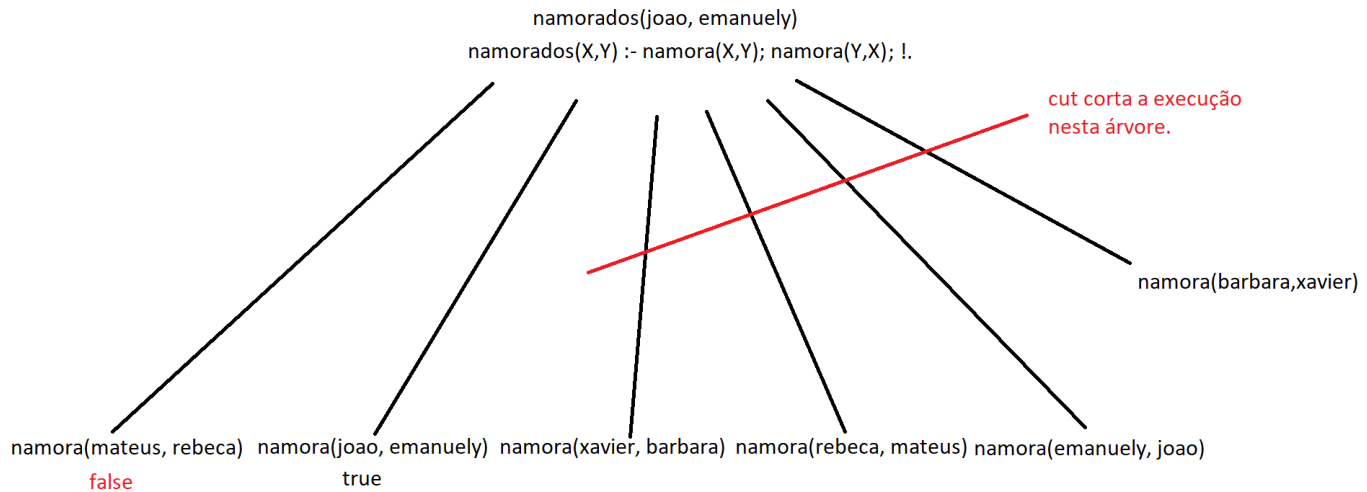




Matrícula

Nome

Como prolog começa pela esquerda, prolog agora tenta resolver `namorados(joao, emanuely)`:



O cut corta a checagem a partir de `trai(xavier)`, e portanto não é necessário realizar o resto da avaliação. Esperamos obter o resultado `X = joao` para a pergunta `tatraindo(X, emanuely)`. A subárvore `namorados(xavier, emanuely)` **não** será percorrida, bem como as subárvores a partir de `namora(xavier, barbara)` na resolução de `namorados(joao, emanuely)`.



Matrícula	Nome
-----------	------

Questão 22 – Construa quatro programas em Hugs para explicar o funcionamento de: expressão let-in, expressão where, função anônima, e comando guardado (0,5 para cada programa). Não use exemplos de sala de aula. [2,0]

Justificativa das questões respondidas como falso.

2) Relatórios elaborados são mais destinados a aplicação comercial, e não para a aplicação científica. Portanto, falsa.

3) Falsa devido o critério de legibilidade estar associado a manutenibilidade (é mais fácil realizar a manutenção de um código legível do que um ilegível) e não a produtividade.

5) C somente possui checagem de tipo estática, não dinâmica. Afinal, é compilada e não interpretada.

8) Falsa, pois a ordem de execução é definida previamente independentemente de um conjunto de conflito ou não. Para Prolog, por exemplo, é realizada através da busca em profundidade à esquerda.

9) Errada pois linguagens com interpretação pura não necessitam converter código fonte em arquivo intermediário. Por exemplo o Javascript, que executa linha a linha interpretando diretamente. Java gera código intermediário (converte source em bytecode), e portanto não é interpretação pura.

11) Hugs não suporta funções globais. Pelo que me lembro, uma das justificativas pela documentação do Prolog seria a de que funções (ou variáveis) globais são um mau hábito, e tudo deveria ser passado através de parâmetros, pois deixa o código mais legível.

13) Este erro só apareceria caso fosse feita a tentativa de calcular 1 dividido por 0. Porém, como é utilizada a avaliação preguiçosa, Hugs determina que qualquer x passado a função **zero x** retornará 0, e nem sequer tenta realizar 1/0.

17) A avaliação preguiçosa somente diz que um valor só será interpretado/calculado quando for utilizado, e não de que caso seja utilizado em mais de um lugar deverá ser mantido em cache (“cotizada”).

18) Prolog utiliza encadeamento de regras para trás, e não para a frente.

19) $((^3).(/2))$ é uma composição, como na matemática a função composta $f(g(x))$. Neste caso, $g(x)$ seria divisão por 2, e elevar a 3 seria o $f(x)$. Aplicando, obtemos $((10 / 2)^3)$, que resulta em $5^3 = 125$, e não 500.

20) A função declara logo no comecinho que o resultado terá cabeça 1, com cauda sendo uma nova lista de cabeça 1 e cauda `[x+y | (x,y) <- (zip fiblst (tail fiblst))]`. Portanto, é esperado que a resposta seja `[1,1,2,3,5]` e não `[1,2,3,5,8]`.